examine how you can use this information in your development of a Dapp.

CODING THE DAPP APPLICATION

Take the time to study the entire app.js code, because you'll have to code a specific one (app.js) for every Dapp. You can use listing 7.2 as the basis for coding app.js in future Dapp development. When you want to code an app.js for a Dapp, armed with the knowledge of web3 in this chapter, you can use the following guidelines:

- Define data for initialization.
- Instantiate the web3 object, and initialize it with a web3 provider. In this chapter, the web3 provider is Ganache; in future chapters, it will be other Ethereum client nodes.
- Link the contract, using its ABI (.json file) and the address at which it is deployed.
- Code the access to the smart contract functions and interaction with the (web) UI.
- Add any supporting functions to facilitate Dapp testing and demonstration. In listing 7.2, `populateAddress()` is such a function, displaying the account addresses and their balances in the UI.

You've completed your exploration of the new Dapp introduced in this chapter, MPC, and the side channel concept.

7.4.9 Extensions of MPC

Unlike the digital democracy (Ballot in chapter 4), marketplace (ASK in chapter 6), and online auction (blind auction in chapter 6) models, the micropayment channel model you learned in this